Probabilistic Programming

Modern Software for Bayesian Modelling

Edward Milsom

2026-10-15

Slide 1 In this Episode ...

- Quick recap of Hamiltonian Monte Carlo (HMC)
  - demo, using a hand-coded HMC implementation
- Applying HMC in practice
  - software library options
  - introduction to `numpyro`
- A new paradigm: probabilistic programming
  - step-by-step walk-through of basic use
  - another demo
- Looking ahead to hierarchical probabilistic graphical models

Hamiltonian Monte Carlo

Slide 2 Recap: Hamiltonian Monte Carlo (HMC)

This is the "state of the art" in MCMC (and stochastic approximation)

- It exploits gradient information from $P^*(\boldsymbol{\theta})$: $\frac{\partial \log P^*(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}}$
- Introduces momentum variables $\rho_i = \partial\theta_i/\partial t$
- Embeds $P^*(\boldsymbol{\theta})$ within a Hamiltonian function:

$$H(\boldsymbol{\theta}, \boldsymbol{\rho}) = -\log P^*(\boldsymbol{\theta}) + \frac{1}{2} \| \boldsymbol{\rho} \|^2$$

- A new state $\boldsymbol{\theta}'$ is proposed by randomly sampling $\boldsymbol{\rho}$ then iterating (numerically) the Hamiltonian dynamics ahead in time by several steps
- Metropolis criterion is then applied to accept/reject the new state with probability

$$\min\left\{1, \exp\left[H(\boldsymbol{\theta}, \boldsymbol{\rho}) - H\left(\boldsymbol{\theta}', \boldsymbol{\rho}'\right)\right]\right\}$$

- The new state should be independent (no sub-sampling)
- Intuition: kicking a ball around the $\log P^*(\boldsymbol{\theta})$ landscape

---

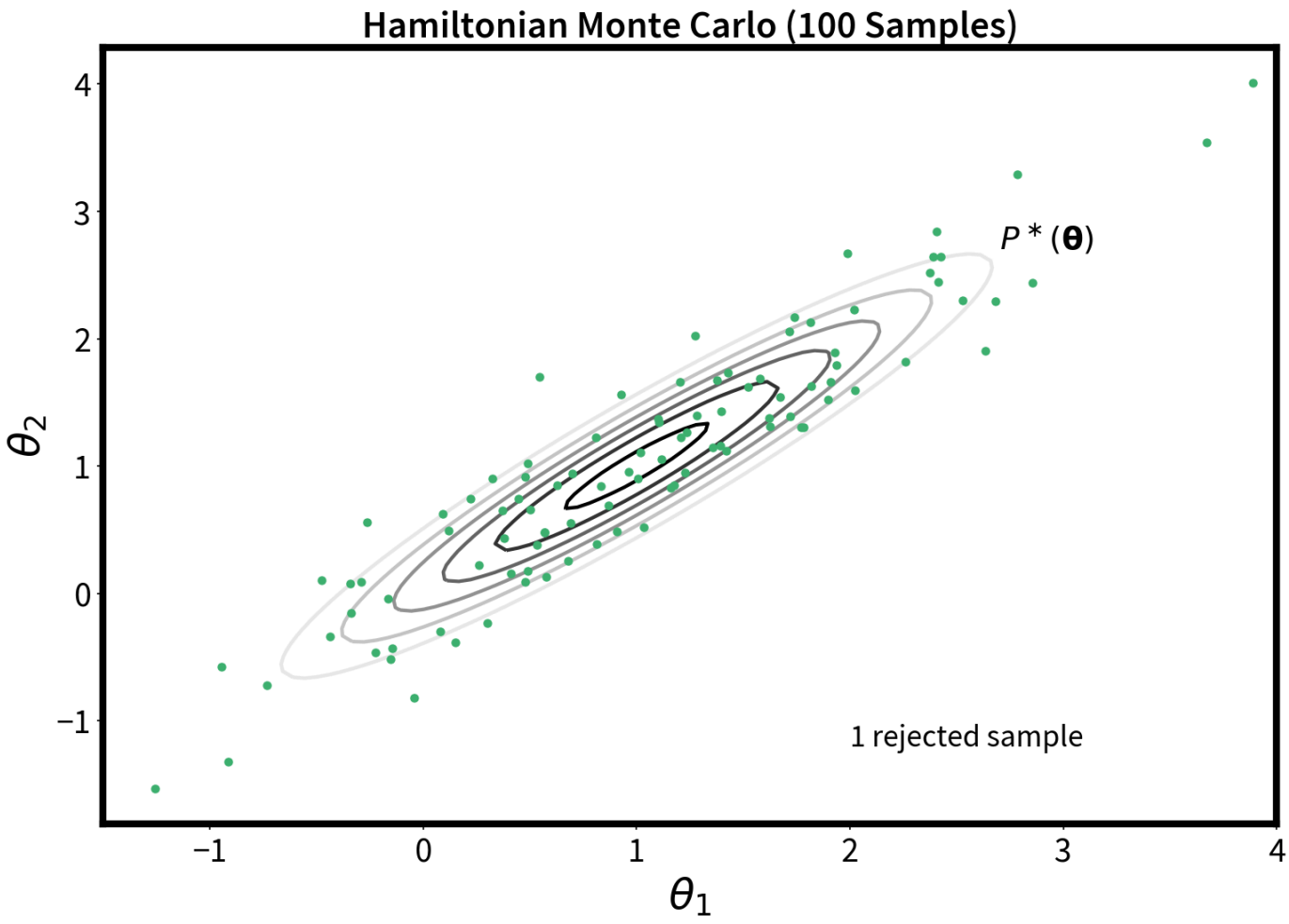
Slide 3 Want to Know More About How HMC Works?

There are two excellent introductory papers, with plenty of accessible detail:

- "MCMC using Hamiltonian Dynamics", Radford M. Neal [https://arxiv.org/abs/1206.1901]

- “A Conceptual Introduction to Hamiltonian Monte Carlo”, M. Betancourt [https://arxiv.org/abs/1701.02434]

Slide 4 Hamiltonian Monte Carlo — Correlated Gaussian

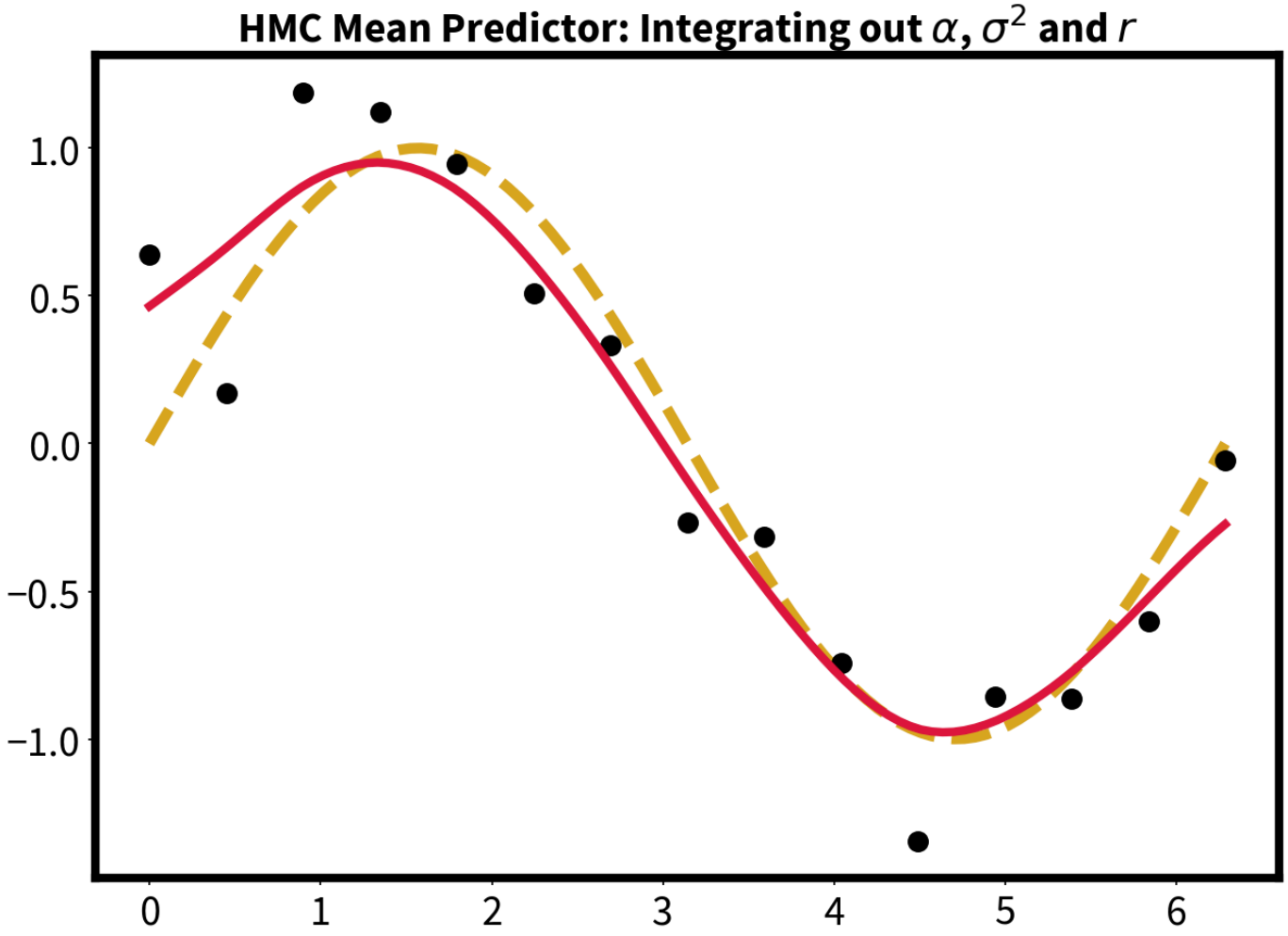


Slide 5 Demo

- Sampling from a Gaussian (and another distribution) with a hand-coded HMC implementation ...

Slide 6 Hamiltonian Monte Carlo — Linear Regression

The mean predictor, averaged over $p\left(\alpha, \sigma^2, r|\mathbf{y}\right)$



Slide 7 Hamiltonian Monte Carlo — Things to Note

- HMC can be very effective, but ...
- ... there is some “voodoo” regarding the implementation:
  - how many steps of dynamics simulation L?
  - what simulation step size (time-step) ϵ?

- "simulation time" $\tau = L \times \epsilon$, the "distance travelled" across the $\log\ p(\boldsymbol{\theta})$ landscape
- large ϵ means less accurate dynamics (more rejected samples)
- small random perturbations to ϵ?
- re-sampling of momentum $\boldsymbol{\rho}$, and use of momentum "persistence"?
- random inversion of direction of "time"?
- what is the desired acceptance rate? 65%?

- These factors may require some tuning to the task at hand
- Alternatively, we can use a software library implementation of HMC that incorporates the NUTS (No U-turn Sampler) kernel

---

Slide 8 Software for Bayesian Inference & MCMC?

There is no MCMC software in the 'standard' *Python* data science distributions, but to apply HMC some other libraries to seriously consider are:

- `Stan` (interfaces with *Python* via PyStan) `https://pystan.readthedocs.io`



- `PyMC` (native *Python*) `http://pymc.io/`



- `numpyro` (native *Python*) `https://num.pyro.ai`



Probabilistic Programming

Slide 9 A Quick Look at ... Probabilistic Graphical Model Representation

- When thinking probabilistically, it can be helpful to visualise our models using probabilistic graphical model (PGM) format



- This specifies a joint distribution that factorises thus:

$$p\left(y, \mathbf{x}, \theta\right) = p\left(y|\mathbf{x}\right) p\left(\mathbf{x}|\theta\right) p\left(\theta\right)$$

  - shaded nodes (y) are observed

  - clear nodes ($\mathbf{x}$, θ) are unknowns, or latent variables
  - arrows indicate conditional dependencies
- We'll return to this in more detail in the context of hierarchical models (Week 7)
- For now, this is a useful context for probabilistic programming ...

Slide 10 What is "Probabilistic Programming"?

- A "model" is defined declaratively, within a function (such as in *Python*)
- Generally this will include defining the (conditional) distributions of latent variables, priors and observation models (likelihoods)
- This function is then treated as a pseudo-object which can be passed to an inference engine for various manipulations
  - sampling from the prior
  - sampling from the posterior, given observation data
  - making predictions
- It enables bespoke models, especially those derived from PGMs, to be rapidly implemented, saving a huge amount of manual coding …

Slide 11 Simple Bayesian Model: Bernoulli with Beta Prior

- The probability density function (p.d.f.) for a single trial:
$$P\left(y|\theta\right) = \theta^{y}\left(1-\theta\right)^{1-y}$$
where $y \in \{0, 1\}$ and θ is probability of "success" $(y = 1)$

- Assume the data, $\mathcal{D} = \{y_1, y_2, ..., y_N\}$, is i.i.d. so:
$$P\left(\mathcal{D}|\theta\right) = \theta^{n_1}\left(1-\theta\right)^{n_0}$$
where n_0, n_1 denote the number of failures/successes, with $n_0 + n_1 = N$

- The conjugate prior for θ in the Bernoulli is the Beta distribution:
$$p_0\left(\theta|\alpha,\beta\right) = \frac{\Gamma\left(\alpha+\beta\right)}{\Gamma\left(\alpha\right)\Gamma\left(\beta\right)}\theta^{\alpha-1}\left(1-\theta\right)^{\beta-1}$$

- The hyper-parameters $\alpha > 0$ and $\beta > 0$ are often referred to as "prior sample sizes" or "pseudo-counts"

Slide 12 Probabilistic Programming in Numpyro

```
import numpyro
import numpyro.distributions as dist
from numpyro.infer import MCMC, NUTS, Predictive
import jax
import jax.numpy as jnp
import arviz as az
```

We can then:

1. Define models (specify joint distributions)

2. Render them as a PGM (optional)

3. Sample from the prior (think generatively!)

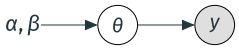
4. Sample from the posterior given observations (and maybe predict)

5. Diagnose and visualise

Slide 13 1. Define Model: Bernoulli with Beta Prior

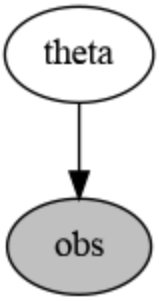
This is the main event!

```
        def model_bernoulli(hypers, y):
            theta = numpyro.sample("theta", dist.Beta(hypers[1],
hypers[0]))
            numpyro.sample("obs", dist.BernoulliProbs(theta),
obs=y)
```



Slide 14 2. Render the Model

```
        dummy_hyper = jnp.ones(2)
        dummy_data = jnp.ones(1)
        numpyro.render_model(model_bernoulli, model_args=
(dummy_hyper, dummy_data))
```



This is *very* useful functionality, but requires an additional software install — see *Moodle*.

Slide 15 3. Sample from the Prior

```
        beta_hyper = jnp.array([1, 1])
        predict = Predictive(model_bernoulli, num_samples=5)
        rng_key = jax.random.key(seed=0)
        samples = predict(rng_key, hypers=beta_hyper, y=None) #
Note we pass y=None to sample from the prior predictive distribution
        print("Theta:", samples["theta"])
        print("Observations:", samples["obs"])
```

Output

```
Theta: [0.21130666 0.31980485 0.3934319  0.5034159  0.66317576]
Observations: [1 0 0 1 0]
```

Slide 16 4. Sample from the Posterior

Make data

```
num_data = 25
rng_key = jax.random.key(seed=0)
y_train = jax.random.bernoulli(rng_key, p=0.7, shape=
(num_data,))
print(f"Empirical theta: {sum(y_train)/ num_data:.3f}")
Empirical theta: 0.600
```

Run NUTS (HMC) sampler

```
kernel = NUTS(model_bernoulli)
mcmc = MCMC(kernel, num_samples=1000, num_warmup=500)
rng_key = jax.random.key(0)
mcmc.run(rng_key, hypers=beta_hyper, y=y_train)
```

Run-time output

```
sample: 100
```

Slide 17 5a. Diagnose

Summary diagnostics

```
mcmc.print_summary()
```

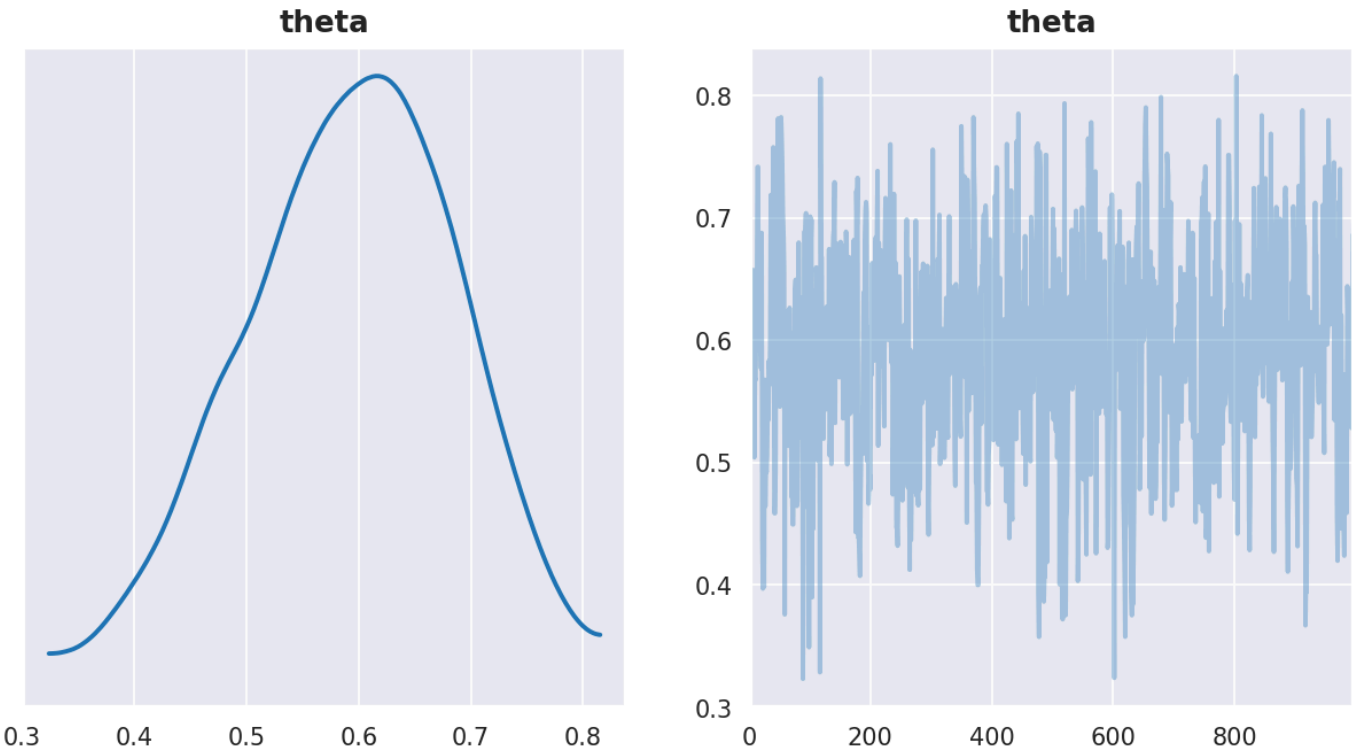
Output

```
          mean       std    median      5.0                                
theta      0.59      0.09      0.60      0.45      0.74    471.81
1.00
```

```
Number of divergences: 0
```

Slide 18 5b. Visualise Use “arviz”

```
data = az.from_numpyro(mcmc)
az.plot_trace(data, figsize=(10,8))
```



Slide 19 5a. (again): What did those numbers mean?

```
           mean       std    median       5.0                                  
theta      0.59      0.09      0.60      0.45      0.74    471.81
1.00

Number of divergences: 0
```

- `mean`, `std`, `median` are posterior mean, standard deviation and median of θ.
- `5.0%` and `95.0%` are the 5th and 95th percentiles of the posterior of θ.
- `n_eff` is the effective sample size (highly correlated samples reduce the effective sample size, which is bad).
- `r_hat` is the Gelman-Rubin statistic. It compares the between-chain and within-chain variance to assess convergence (should be close to 1).
- `Number of divergences` indicates how many times the sampler encountered numerical issues (should be 0 for a well-behaved model).

Slide 20 Quick Live Run-through

- Back to *Jupyter* ...

Slide 21 Summary, and Looking Ahead

- Probabilistic Programming represents a different way to think about model implementation, while beneficially:
  - abstracting away the implementation detail
  - seamlessly incorporating stochastic approximation
- It harmonises with the probabilistic graphical model (PGM) representation
  - we will look at this properly in Week 7 (Lectures 11 and 12)

- It is especially convenient when used to implement hierarchical models
    - this is the primary focus of Week 7
- We still require care when running MCMC/HMC/NUTS, particularly with the parameterisation of our models